

Appening AI Assignment Refernce material

Candidate Reference Guide: RAG-based AI Chatbot Project

This reference document provides a complete guide to help you build a Retrieval-Augmented Generation (RAG) AI chatbot using **Python**, **LangGraph**, **Pinecone**, and **FastAPI / Streamlit**.

1. Objective

The goal of this assignment is to develop a RAG-based AI Chatbot that strictly answers questions based on a provided knowledge source: the **Agentic AI eBook**

(https://drive.google.com/file/d/15VLphKcY23_fpYxN62UEQRri_psRVfP9/view?usp=sharing).

Core Goals:

- **Ingest & Store:** Parse the PDF document, divide it into text chunks, convert chunks into embedding vectors, and index them inside Pinecone.
- **Orchestrate Workflow:** Utilize **LangGraph** to build a stateful execution graph that handles retrieval and grounded response generation.
- **Strict Grounding:** Ensure the chatbot answers queries using **only** information retrieved from the document context.
- **API / UI Output:** Expose a FastAPI backend or Streamlit interface that returns:
 1. The final generated answer
 2. The retrieved context chunks
 3. A relevance or confidence score

2. Technical Requirements

Software & Hardware

- **Python:** 3.10 or higher.
- **Operating System:** macOS, Linux, or Windows (via WSL2 recommended).
- **RAM:** Minimum 8 GB.

API Keys & External Services

- **OpenAI API Key:** For text embeddings (text-embedding-3-small) and the LLM (gpt-4o-mini or gpt-3.5-turbo).
- **Pinecone API Key:** Free tier account for hosting vector index.

Python Libraries & Dependencies

Create a requirements.txt file containing the following core packages:

Plaintext

langchain>=0.2.0

langgraph>=0.1.0

langchain-openai>=0.1.0

langchain-community>=0.2.0

pinecone-client>=3.0.0

pypdf>=4.0.0

fastapi>=0.110.0

uvicorn>=0.28.0

streamlit>=1.32.0

python-dotenv>=1.0.0

3. Step-by-Step Instructions

1.Set Up Environment & Dependencies:

Prerequisites & API Configuration.

1. **Clone/Initialize Repository:** Create a local directory for your project and initialize Git.
2. **Virtual Environment:** Create and activate an isolated Python environment:

Bash

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

1. **Install Packages:** Install dependencies using `pip install -r requirements.txt`.
2. **Environment Variables:** Create a `.env` file containing your API credentials:

env

```
OPENAI_API_KEY=your_openai_api_key
```

```
PINECONE_API_KEY=your_pinecone_api_key
```

```
PINECONE_INDEX_NAME=agentic-ai-index
```

2.Document Ingestion & Vector Storage Pipeline:

ETL Step for the PDF Document.

1. **Download PDF:** Fetch the PDF file from [\[https://drive.google.com/file/d/15VLphKcY23_fpYxN62UEQRri_psRVfP9/view?usp=sharing\]](https://drive.google.com/file/d/15VLphKcY23_fpYxN62UEQRri_psRVfP9/view?usp=sharing) and store it locally under `data/Ebook-Agentic-AI.pdf`.
2. **Text Extraction:** Use `PyPDFLoader` to load document pages into memory.
3. **Text Chunking:** Apply `RecursiveCharacterTextSplitter` to divide the document into manageable chunks (e.g., `chunk_size=1000`, `chunk_overlap=200`).
4. **Pinecone Index Setup:** Initialize a Pinecone index with a dimension of 1536 (matching OpenAI's `text-embedding-3-small` or `text-embedding-ada-002`) using cosine distance.

5. **Upsert Embeddings:** Convert chunks to vectors and store them in Pinecone using LangChain's vector store wrapper.

3. Build RAG Graph with LangGraph:

Stateful Workflow Design.

1. **Define Graph State:** Create a TypedDict class AgentState containing fields like question, context, answer, and score.
2. **Define Nodes:**
 - **retrieve Node:** Queries Pinecone for top- k relevant chunks using the user prompt.
 - **generate Node:** Formulates a system prompt requiring the LLM to rely strictly on the retrieved context chunks and compute/assign a confidence score.
3. **Assemble StateGraph:**
 - Define starting node (START -> retrieve).
 - Route retrieve -> generate -> END.
 - Compile the graph using workflow.compile().

4. Expose API / UI Interface:

Interaction Layer.

1. **FastAPI Route:**
 - Create a POST endpoint /chat accepting a JSON payload with a query string.
 - Invoke the compiled LangGraph execution graph with the incoming question.
 - Format and return a JSON payload containing answer, retrieved_chunks, and confidence_score.
2. **Streamlit UI (Optional Alternative):**
 - Build a lightweight web UI featuring a chat input and a side panel displaying the retrieved context chunks and score.

5. Verification & Testing:

Quality Assurance.

Run 5–6 benchmark queries against your API to verify response quality and context retrieval:

1. *"What is Agentic AI according to the eBook?"*
2. *"How do AI agents differ from traditional automation systems?"*
3. *"What are the core components of an Agentic Architecture?"*
4. *"What role does memory play in Agentic AI workflows?"*
5. *"Who won the 2022 FIFA World Cup?" (Validation test: system should refuse or state that the context lacks this information).*

4. Project Structure

Organize your codebase cleanly using the standard Python directory structure below:

Plaintext

rag-agentic-ai/

|

|— data/

| — Ebook-Agentic-AI.pdf # Downloaded source document

|

|— src/

| |— __init__.py

| |— ingestion.py # PDF loading, splitting & Pinecone index setup

| |— graph.py # LangGraph workflow definition & state logic

| — config.py # Environment setup & constants

|

|— app.py # FastAPI application / Streamlit app

|— requirements.txt # Python project dependencies

|— .env.example # Template for environment variables

|— README.md # Comprehensive deployment instructions

— tests_sample_queries.py # Script containing 5-6 sample test queries

5. Example Code Snippets

The snippets below demonstrate key structural patterns for the implementation.

Snippet 1: Document Ingestion (src/ingestion.py)

Python

```
from langchain_community.document_loaders import PyPDFLoader
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
from langchain_openai import OpenAIEmbeddings
```

```
from langchain_pinecone import PineconeVectorStore
```

```
import os
```

```
def run_ingestion(pdf_path: str, index_name: str):
```

```
    # 1. Load document
```

```

loader = PyPDFLoader(pdf_path)
docs = loader.load()

# 2. Chunk document
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = text_splitter.split_documents(docs)

# 3. Create embeddings & save to Pinecone
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = PineconeVectorStore.from_documents(
    documents=chunks,
    embedding=embeddings,
    index_name=index_name
)
return vector_store

```

Snippet 2: LangGraph State & RAG Workflow (src/graph.py)

Python

```

from typing import List, TypedDict

from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_pinecone import PineconeVectorStore

class AgentState(TypedDict):
    question: str
    context: List[str]
    answer: str
    score: float

```

```

def build_rag_graph(index_name: str):

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = PineconeVectorStore(index_name=index_name, embedding=embeddings)
    retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    # Define Nodes

    def retrieve_node(state: AgentState):
        docs = retriever.invoke(state["question"])
        context_texts = [d.page_content for d in docs]
        return {"context": context_texts}

    def generate_node(state: AgentState):
        context_str = "\n\n".join(state["context"])

        prompt = f"""You are a strict assistant. Answer the question relying ONLY on the context below.
        If the context does not contain enough info, state 'I cannot answer based on the provided
        document.'

        Context:

        {context_str}

        Question: {state['question']}"""

        response = llm.invoke(prompt)

        # Simple score heuristic based on retrieved context availability
        confidence = 0.95 if len(state["context"]) > 0 else 0.0

        return {"answer": response.content, "score": confidence}

    # Build Graph

```

```
workflow = StateGraph(AgentState)
workflow.add_node("retrieve", retrieve_node)
workflow.add_node("generate", generate_node)
```

```
workflow.add_edge(START, "retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)
```

```
return workflow.compile()
```

Snippet 3: FastAPI Interface (app.py)

Python

```
from fastapi import FastAPI
from pydantic import BaseModel
from src.graph import build_rag_graph
import os

app = FastAPI(title="Agentic AI RAG API")
graph = build_rag_graph(index_name=os.getenv("PINECONE_INDEX_NAME", "agentic-ai-index"))

class QueryRequest(BaseModel):
    query: str

class QueryResponse(BaseModel):
    final_answer: str
    retrieved_context: list[str]
    confidence_score: float

@app.post("/chat", response_model=QueryResponse)
async def chat_endpoint(request: QueryRequest):
    initial_state = {"question": request.query, "context": [], "answer": "", "score": 0.0}
    result = graph.invoke(initial_state)
```

```
return QueryResponse(  
    final_answer=result["answer"],  
    retrieved_context=result["context"],  
    confidence_score=result["score"]  
)
```

6. Submission Guidelines

To successfully submit your completed task, adhere strictly to the following instructions:

- **Deadline:** Within **24 hours (1 day)** from the time of email receipt.
- **Submission Method:** Submit **only** your GitHub repository URL via the requested channel/reply email.
- **Repository Visibility:** Ensure the GitHub repository permissions are set to **Public** (or explicitly shared with the review team's handles).
- **Submission Checklist:**
 - [] Working RAG implementation in Python (No-code / low-code / vibe-coding tools strictly avoided).
 - [] Well-structured README.md with explicit setup, ingestion, and execution commands.
 - [] API or UI output returning answer, context chunks, and confidence score.
 - [] 5–6 test queries demonstrating document grounding.
 - [] Clean architecture breakdown included in the README.